

Oracle Database 19c Backup & Recovery · 제1장

백업·복구 개요와 19c 복구 아키텍처

장애 유형 · 복구 대상 파일 · SCN과 리두 · 인스턴스 복구 vs 미디어 복구 · ADR

학습 목표 (Objectives)

- 이 장을 마치면 다음을 수행할 수 있어야 한다.
 - 데이터베이스 장애의 유형을 구분하고 유형별 복구 작업의 성격을 설명한다
 - 복구에 관여하는 물리 파일의 역할과 유실 증상을 설명한다
 - SCN·리두·언두·체크포인트가 복구에서 어떻게 쓰이는지 설명한다
 - 인스턴스 복구와 미디어 복구를 구분하고 완전/불완전 복구를 판별한다
 - alert log와 ADR에서 장애 진단의 출발점을 찾는다
- 다섯 목표는 각각 1.1 / 1.2 / 1.3 / 1.4~1.5 / 1.6 절과 대응된다.

이 장의 구성 (Agenda)

- 1.1 백업과 복구의 목적 — 장애 유형 · RPO/RTO · 솔루션 지도 · 백업 분류
- 1.2 복구에 관여하는 물리 파일 — 데이터/제어/리두/아카이브/파라미터
- 1.3 변경의 흐름 — SCN · 리두와 언두 · 체크포인트 · 로그 스위치
- 1.4 인스턴스 복구 — 롤포워드와 롤백 · MTTR 조절
- 1.5 미디어 복구와 복구 유형 — 완전/불완전 · 모드별 복구 범위
- 1.6 진단 정보 — Alert Log와 ADR
- 1.7 퀴즈 · 1.8 요약 · 1.9 실습 1 개요

참고:

이 장은 개념 조망 장이다. 파일을 손상시키거나 복구하는 명령은 4장부터 등장한다.

1.1

백업과 복구의 목적

- 장애 유형 6가지 중 백업본이 필요한 것은 미디어 실패뿐
- 복구 설계의 출발점은 RPO와 RTO
- User Managed · RMAN · Flashback · Data Pump는 보완 관계

장애 유형 6가지

- 명령문 실패 — 문법 오류, 권한 부족, 테이블스페이스 공간 부족
- 사용자 프로세스 실패 — 클라이언트 비정상 종료, PMON이 자원 정리
- 네트워크 실패 — 리스너/네트워크 경로 단절, 데이터 손상 없음
- 사용자 오류 — 잘못된 DML·DROP, 문법은 정상이나 의도가 잘못됨
- 인스턴스 실패 — 전원 차단, SHUTDOWN ABORT
- 미디어 실패 — 디스크 손상, 파일 삭제

유형별로 DBA가 하는 일

- 앞의 네 유형(명령문·프로세스·네트워크·사용자 오류)
 - 파일 복원을 동반하지 않는다
 - 사용자 오류만 별도 경로가 있다 — Flashback 계열(18장)
- 인스턴스 실패
 - Oracle이 STARTUP 시 자동 복구, DBA는 결과만 확인
- 미디어 실패
 - 백업본이 반드시 필요하다
 - DBA가 RESTORE와 RECOVER를 직접 수행한다

실무 시나리오:

'DB가 안 됩니다' 연락을 받으면 순서는 `v$instance STATUS` → `v$database OPEN_MODE` → alert log 마지막 수집 줄이다. 이 세 단계로 인스턴스 문제인지 파일 문제인지가 갈린다.

복구 목표 — RPO와 RTO

- RPO(Recovery Point Objective) — 어느 시점까지의 데이터를 지킬 것인가
 - 허용 가능한 데이터 손실량으로 표현
 - RPO 0 → ARCHIVELOG 모드와 리두 다중화가 최소 요건
 - RPO 하루 → 야간 전체 백업만으로 충족, NOARCHIVELOG 운영 가능
- RTO(Recovery Time Objective) — 얼마 만에 서비스를 재개할 것인가
 - 허용 가능한 중단 시간으로 표현
 - 백업 방식보다 복원 대상의 크기가 좌우한다

RTO를 줄이는 법 · MTTR과의 구분

- 1TB 전체 복원 시간은 디스크 대역폭이 결정한다
 - 백업 방식만 바뀌서 줄일 수 있는 폭은 제한적이다
- RTO를 줄이려면 복원량 자체를 줄여야 한다
 - Image Copy + SWITCH — 복원 없이 복사본으로 전환(14장)
 - 증분 백업 — 변경분만 적용(14장)
 - Block Media Recovery — 손상 블록만 복구(16장)
- MTTR은 인스턴스 복구에 한정된 개념이다
 - FAST_START_MTTR_TARGET은 미디어 복구 시간과 무관하다

백업·복구 솔루션 지도

- User Managed Backup — OS 명령(cp, tar)으로 파일 직접 복사 (3~12장)
- RMAN — Oracle 전용 백업 도구, 블록 검증·증분·자동 조인 (13~16장)
- Flashback — 백업본 없이 언두/플래시백 로그로 과거 복원 (18장)
- Data Pump — 논리적 객체 단위 내보내기/들여오기 (19장)
- 네 가지는 대체가 아니라 보완 관계다
 - 미디어 실패 → User Managed 또는 RMAN만 가능
 - 사용자 오류 → Flashback이 압도적으로 빠름
 - 버전 간 이관·스키마 이동 → Data Pump만 가능

왜 User Managed를 먼저 배우는가

- RMAN의 RESTORE와 RECOVER는 내부적으로 User Managed 절차와 같은 일을 한다
- 손으로 한 번 해 본 사람만이 알 수 있는 것
 - RMAN이 무엇을 대신해 주는지
 - 실패했을 때 어디를 봐야 하는지
- 이 과정의 순서: User Managed(3~12장) → RMAN(13~16장)
- NOARCHIVELOG(4~7장)를 먼저 다루는 이유도 같다
 - 그것을 권장해서가 아니다
 - 리두가 없을 때 무엇을 잃는지 겪어야 ARCHIVELOG의 가치가 분명해진다

백업의 분류 — 세 개의 축

- 범위: 전체 백업 vs 부분 백업
 - 부분 백업은 ARCHIVELOG 모드에서만 의미가 있다
- 일관성: 일관성 있는 백업 vs 일관성 없는 백업
 - 모든 파일의 SCN이 일치하면 일관성 있는 백업 — 정상 종료 후에만 가능
 - 일관성 없는 백업은 복원 후 반드시 리두 적용이 필요하다
- 상태: 오프라인(cold) vs 온라인(hot)
 - 온라인 백업은 ARCHIVELOG 모드에서만 가능하다

NOARCHIVELOG에서 가능한 조합은 하나뿐

- NOARCHIVELOG에서 가능한 백업 = 전체 + 일관성 있는 + 오프라인
 - 부분 백업 불가 — 리두가 없어 개별 파일을 최신으로 끌어올릴 수 없다
 - 온라인 백업 불가 — 백업 중 변경을 복구할 리두가 없다
- SHUTDOWN ABORT 후 받은 백업은 오프라인이지만 일관성이 없다
 - NOARCHIVELOG 환경에서는 사용할 수 없는 백업이 된다
 - 3장에서 실제로 확인한다

참고:

Data Guard와 GoldenGate는 재해 복구(DR) 솔루션으로 별도 과정에서 다룬다. 다만 17장의 DUPLICATE는 Data Guard 대기 DB 구성의 기반 기술이다.

1.2

복구에 관여하는 물리 파일

- 복구는 '무엇이 없어졌고 무엇으로 되살리는가'의 문제
- 파일마다 유실 증상과 복구 경로가 다르다
- 제어 파일과 리두 로그는 다중화로 유실 자체를 예방

데이터 파일 (Data File)

- 실제 데이터가 저장되는 파일, 관례적으로 .dbf 확장자
- 하나의 테이블스페이스 = 하나 이상의 데이터 파일
- 각 파일 헤더에 체크포인트 SCN이 기록된다 — 복구 필요 판단의 근거
- 유실 증상은 어느 테이블스페이스인지에 따라 크게 다르다
 - SYSTEM / UNDO — 인스턴스가 기동되지 않거나 즉시 중단
 - 일반 사용자 TS — DB는 열린 채, 해당 객체 접근 시에만 오류
 - SYSAUX — AWR·통계 등 부가 기능만 영향, 업무 트랜잭션은 계속

데이터 파일 유실 시 나타나는 ORA 오류

- ORA-01110 — data file 번호와 이름을 알려 준다
 - 단독으로 나오지 않고 원인을 알려 주는 오류와 짝을 이룬다
- ORA-01157 — 파일을 식별.잠글 수 없다 (파일이 아예 없을 때)
- ORA-01113 — 복구가 필요하다 (백업본으로 되돌린 직후)
- ORA-01578 — 블록 단위 손상 (파일은 있으나 내부 블록이 깨짐)
- 오류 번호가 곧 진단이다 — 어떤 조치가 필요한지가 번호로 갈린다

제어 파일 (Control File)

- 데이터베이스의 물리 구조와 상태를 기록하는 작은 바이너리 파일
- 담고 있는 정보
 - DB 이름·생성 시각·DBID
 - 모든 데이터 파일과 리두 로그 파일의 이름·경로·크기
 - 현재 로그 시퀀스 번호와 체크포인트 정보
 - 아카이브 로그 이력, RMAN 백업 메타데이터
- MOUNT 단계에서 읽힌다 — 없으면 마운트 불가, ORA-00205
- 복구 경로 2가지: 백업본 복사 / 트레이스로 재생성 (6장·11장)

제어 파일 위치와 내용 확인

V\$CONTROLFILE / V\$CONTROLFILE_RECORD_SECTION — 다중화 상태와 레코드 사용량

```
-- 제어 파일 위치와 다중화 상태 확인
```

```
SYS@orcl> SELECT name FROM v$controlfile;
```

```
NAME
```

```
-----  
/u01/app/oracle/oradata/ORCL/control01.ctl
```

```
/u01/app/oracle/oradata/ORCL/control02.ctl
```

```
-- 제어 파일이 담고 있는 레코드 종류와 사용량
```

```
SYS@orcl> SELECT type, records_total, records_used
```

```
2 FROM v$controlfile_record_section
```

```
3 WHERE type IN ('DATAFILE', 'REDO LOG', 'ARCHIVED LOG');
```

```
TYPE                RECORDS_TOTAL RECORDS_USED
```

```
-----  
REDO LOG                16                3
```

```
DATAFILE                100                4
```

```
ARCHIVED LOG            386               98
```

온라인 리두 로그 (Online Redo Log)

- 데이터베이스의 모든 변경을 시간 순으로 기록
- 최소 2개 그룹, 각 그룹은 1개 이상의 멤버(동일 내용 사본)
- 그룹은 순환 사용 — 가득 차면 다음 그룹으로 전환(로그 스위치)
- 그룹의 세 가지 상태
 - CURRENT — LGWR가 현재 기록 중, 정확히 하나만 존재
 - ACTIVE — 기록 완료, 체크포인트 미완료, 인스턴스 복구에 필요할 수 있음
 - INACTIVE — 체크포인트 완료, 인스턴스 복구에 더는 불필요

리두 로그 상태가 복구를 가르다

- INACTIVE 그룹 유실
 - CLEAR LOGFILE로 재생성 가능, 데이터 손실 없음
- CURRENT 그룹 유실
 - 아직 데이터 파일에 반영되지 않은 변경이 함께 사라진다
 - 완전 복구가 불가능해진다 — 불완전 복구 후 RESETLOGS
- 그래서 멤버를 둘 이상 두어 다중화한다
 - 그룹 안의 멤버 하나가 깨져도 다른 멤버로 계속 동작
 - MEMBERS 컬럼이 1이면 운영 환경에서는 즉시 시정 대상

실무 시나리오:

v\$logfile의 STATUS가 INVALID인 멤버가 있는지도 함께 확인한다. 비어 있으면 정상이다.

리두 로그 상태 조회

V\$LOG / V\$LOGFILE — 그룹 상태와 멤버 구성 확인

-- 리두 로그 그룹의 상태.시퀀스.아카이브 여부

```
SYS@orcl> SELECT group#, sequence#, bytes/1024/1024 AS mb, members, archived, status  
2 FROM v$log ORDER BY group#;
```

GROUP#	SEQUENCE#	MB	MEMBERS	ARC	STATUS
1	97	10	1	YES	INACTIVE
2	98	10	1	NO	CURRENT
3	96	10	1	YES	INACTIVE

-- 그룹별 멤버(파일) 목록 - 다중화 상태 확인

```
SYS@orcl> SELECT group#, member, status FROM v$logfile ORDER BY group#;
```

GROUP#	MEMBER	STATUS
1	/u01/app/oracle/oradata/ORCL/redo01.log	
2	/u01/app/oracle/oradata/ORCL/redo02.log	
3	/u01/app/oracle/oradata/ORCL/redo03.log	

아카이브 리두 로그 (Archived Redo Log)

- 온라인 리두 로그가 덮어쓰이기 전에 별도 위치로 복사해 둔 파일
- ARCHIVELOG 모드에서만 생성, ARCn 프로세스가 복사 담당
- 과거 백업본에서 현재까지의 변경을 순서대로 적용할 수 있게 한다
 - 완전 복구의 전제 조건
- NOARCHIVELOG 모드에서는 생성되지 않는다
 - 온라인 리두에 남아 있는 범위 안에서만 복구 가능
 - 그 범위를 벗어나면 백업 시점으로 되돌아가는 수밖에 없다
- LOG_ARCHIVE_DEST_n으로 여러 곳에 동시 기록 가능

아카이브 모드와 SCN 구간 확인

ARCHIVE LOG LIST / V\$ARCHIVED_LOG — 각 로그가 담는 SCN 범위

-- 아카이브 모드와 대상 확인

```
SYS@orcl> ARCHIVE LOG LIST
```

Database log mode	Archive Mode
Automatic archival	Enabled
Archive destination	/home/oracle/arch1
Oldest online log sequence	96
Next log sequence to archive	98
Current log sequence	98

-- 생성된 아카이브 로그의 SCN 범위

```
SYS@orcl> SELECT sequence#, name, first_change#, next_change#  
2 FROM v$archived_log WHERE sequence# >= 96 ORDER BY sequence#;
```

SEQUENCE#	NAME	FIRST_CHANGE#	NEXT_CHANGE#
96	/home/oracle/arch1/arch_1_96_1200585.arc	2081984	2090995
97	/home/oracle/arch1/arch_1_97_1200585.arc	2090995	2097189

파라미터 파일 · 패스워드 파일 · 백업 대상 정리

- SPFILE — 초기화 파라미터 바이너리 파일, \$ORACLE_HOME/dbs/spfile<SID>.ora
 - CREATE PFILE ... FROM SPFILE로 텍스트 사본을 언제든지 확보
 - 유실 시 기동 불가하나 데이터는 무사, 백업본이나 alert log로 재작성 가능
- 패스워드 파일(orapw<SID>) — SYSDBA 원격 접속용, 유실해도 로컬 OS 인증 가능
- TEMP 파일은 백업 대상이 아니다
 - 임시 데이터만 담아 유실 시 재생성으로 끝난다(5장)
- RMAN의 CONTROLFILE AUTOBACKUP을 켜면 SPFILE과 제어 파일이 자동 백업된다(16장)

백업 대상 파일 통합 조회

V\$DATAFILE + V\$CONTROLFILE + V\$LOGFILE — 백업 전 대상 확인

```
-- 백업 대상 물리 파일 전체 목록
```

```
SYS@orcl> SELECT name FROM v$datafile
```

```
2 UNION ALL
```

```
3 SELECT name FROM v$controlfile
```

```
4 UNION ALL
```

```
5 SELECT member FROM v$logfile;
```

```
NAME
```

```
-----  
/u01/app/oracle/oradata/ORCL/system01.dbf  
/u01/app/oracle/oradata/ORCL/sysaux01.dbf  
/u01/app/oracle/oradata/ORCL/undotbs01.dbf  
/u01/app/oracle/oradata/ORCL/users01.dbf  
/u01/app/oracle/oradata/ORCL/control01.ctl  
/u01/app/oracle/oradata/ORCL/control02.ctl  
/u01/app/oracle/oradata/ORCL/redo01.log  
/u01/app/oracle/oradata/ORCL/redo02.log  
/u01/app/oracle/oradata/ORCL/redo03.log
```

```
9 rows selected.
```

1.3

변경의 흐름 — SCN·리두·언두·체크포인트

- SCN은 복구 판단의 기준이 되는 논리적 시계
- 리두는 롤포워드, 언두는 롤백에 쓰인다
- 커밋은 리두 기록만 보장한다

SCN (System Change Number)

- 데이터베이스 내부의 논리적 시계 — 커밋마다 단조 증가
- 시간이 아니라 순서를 나타낸다
- 여러 곳에 기록되며 그 일치 여부가 복구 필요성을 결정한다
 - 제어 파일 — 데이터베이스 전체의 체크포인트 SCN
 - 각 데이터 파일 헤더 — 그 파일의 체크포인트 SCN
 - 리두 로그 — 각 로그가 담는 변경의 SCN 범위
- 모든 값이 같으면 일관성 있는(consistent) 상태
- 어긋나 있으면 그 차이만큼 리두를 적용해야 한다

SCN 비교 — 복구 필요 여부 판단

V\$DATABASE vs V\$DATAFILE — 모든 값이 같으면 일관성 있는 상태

```
-- 제어 파일과 데이터 파일의 체크포인트 SCN 비교
```

```
SYS@orcl> SELECT checkpoint_change# FROM v$database;
```

```
CHECKPOINT_CHANGE#
```

```
-----  
2097189
```

```
SYS@orcl> SELECT file#, name, checkpoint_change#, status FROM v$datafile ORDER BY file#;
```

FILE#	NAME	CHECKPOINT_CHANGE#	STATUS
1	/u01/app/oracle/oradata/ORCL/system01.dbf	2097189	SYSTEM
3	/u01/app/oracle/oradata/ORCL/sysaux01.dbf	2097189	ONLINE
4	/u01/app/oracle/oradata/ORCL/undotbs01.dbf	2097189	ONLINE
7	/u01/app/oracle/oradata/ORCL/users01.dbf	2097189	ONLINE

```
-- SCN을 시각으로 환산
```

```
SYS@orcl> SELECT SCN_TO_TIMESTAMP(2097189) FROM dual;  
02-MAY-25 02.08.31.000000000 AM
```

리두와 언두의 역할 분담

- 리두(redo) — '무엇을 했는가', 변경 후 값을 다시 적용하는 재료
 - 복구 시 롤포워드 사용, 리두 로그 파일에 저장
- 언두(undo) — '무엇이었는가', 변경 전 값으로 되돌리는 재료
 - 롤백·읽기 일관성·Flashback Query에 사용
 - UNDO 테이블스페이스의 데이터 파일에 저장
- 언두 자체의 변경도 리두에 기록된다
 - 언두 테이블스페이스가 손상되어도 리두가 있으면 되살릴 수 있다
- UNDO_RETENTION 기본값 900초 — Flashback Query 가능 범위를 결정(18장)

체크포인트와 세 프로세스

- 체크포인트 = '이 SCN 이전의 변경은 모두 데이터 파일에 반영되었다'는 선언
- 체크포인트 시 수행되는 일
 - DBWn이 dirty buffer를 데이터 파일에 기록
 - CKPT가 제어 파일과 데이터 파일 헤더에 새 SCN 기록
- 프로세스별 역할
 - LGWR — 리두 버퍼 → 리두 로그 파일, 커밋 시 반드시 수행
 - DBWn — 변경 블록 → 데이터 파일, 커밋과 무관하게 자체 판단
 - CKPT — 체크포인트 정보 갱신, 직접 데이터를 쓰지는 않음

커밋이 보장하는 것은 리두뿐

- COMMIT 반환 시점에 보장되는 것 = LGWR의 리두 기록
- 커밋된 데이터가 데이터 파일에 있다는 보장은 없다
- 왜 커밋 시점에 데이터 파일에 쓰지 않는가
 - 커밋마다 해당 블록을 쓰면 디스크 위치가 흩어져 임의 접근이 발생 — 느리다
 - 리두 로그는 순차 기록이라 훨씬 빠르다
 - 데이터 파일 반영은 DBWn이 모아서 처리한다
- 결과: '리두는 최신, 데이터 파일은 과거'가 정상 상태
 - 복구란 그 간격을 메우는 작업이다

SCN 흐름 추적 — 커밋 vs 로그 스위치

커밋만으로는 체크포인트 SCN이 변하지 않는다

-- 변경 → 로그 스위치 → 체크포인트 SCN 변화 추적

```
SYS@orcl> SELECT checkpoint_change# FROM v$database;
```

```
CHECKPOINT_CHANGE#
```

```
-----  
                2097189
```

```
SYS@orcl> UPDATE hr.employees SET salary = salary * 1.1 WHERE employee_id = 100;
```

```
1 row updated.
```

```
SYS@orcl> COMMIT;
```

```
Commit complete.
```

-- 커밋만으로는 체크포인트 SCN이 변하지 않는다

```
SYS@orcl> SELECT checkpoint_change# FROM v$database;
```

```
                2097189
```

-- 로그 스위치가 체크포인트를 유발한다

```
SYS@orcl> ALTER SYSTEM SWITCH LOGFILE;
```

```
System altered.
```

```
SYS@orcl> SELECT checkpoint_change# FROM v$database;
```

```
                2098460
```

로그 스위치

- CURRENT 그룹이 가득 차면 자동 발생, ALTER SYSTEM SWITCH LOGFILE로 강제 가능
- 시퀀스 번호가 1 증가하고, ARCHIVELOG면 직전 그룹이 아카이브 대상이 된다
- 로그 스위치는 체크포인트를 유발한다
 - 체크포인트 완료가 스위치 완료보다 늦을 수 있어 잠시 ACTIVE 상태가 유지된다
- ALTER SYSTEM ARCHIVE LOG CURRENT — 스위치와 아카이브를 함께, 완료까지 대기
 - 백업 직후 실행하면 백업 시점의 리두가 확실히 아카이브된다(3장)

실무 시나리오:

v\$log_history를 시간대별로 집계해 시간당 스위치 횟수를 본다. 통상 3~6회가 적정선이며, 너무 잦으면 리두 파일 크기가 작다는 뜻이다.

1.4

인스턴스 복구 (Instance Recovery)

- 파일은 온전하고 메모리만 유실된 경우
- SMON이 자동 수행 — DBA는 개입하지 않는다
- 롤포워드 후 롤백, 데이터 손실 없음

인스턴스 복구가 필요한 상황

- 인스턴스가 비정상 종료된 뒤 다시 기동할 때 자동 수행
- 비정상 종료에 해당하는 경우
 - SHUTDOWN ABORT
 - 서버 전원 차단, OS 패닉
 - 백그라운드 프로세스 사망으로 인스턴스 자체 중단
- SHUTDOWN NORMAL / IMMEDIATE / TRANSACTIONAL은 해당 없음
 - 종료 전 체크포인트를 완료하므로 복구가 필요 없다
- 자료는 온라인 리두 로그와 언두 — 백업본은 전혀 쓰이지 않는다

롤포워드와 롤백

- 1단계 롤포워드(roll forward) — 리두를 데이터 파일에 적용
 - 마지막 체크포인트 SCN 이후의 모든 리두를 순서대로 적용
 - 커밋된 변경과 커밋되지 않은 변경이 구분 없이 함께 적용된다
 - 끝나면 데이터 파일이 장애 직전 상태와 같아진다
- 2단계 롤백(roll back) — 미커밋 트랜잭션을 언두로 되돌림
 - DB가 열린 뒤에도 백그라운드에서 계속될 수 있다(지연 롤백)
- 왜 롤포워드가 먼저인가
 - 언두 자체가 데이터 파일에 저장되므로, 리두로 복구되어야 신뢰할 수 있다

crash recovery 실제 흐름

SHUTDOWN ABORT 후 STARTUP — alert log에 남는 기록

```
-- 비정상 종료 후 재기동
SYS@orcl> SHUTDOWN ABORT
ORACLE instance shut down.

SYS@orcl> STARTUP
ORACLE instance started.
Database mounted.
Database opened.

-- alert log 발췌
Beginning crash recovery of 1 threads
Started redo scan
Completed redo scan
  read 412 KB redo, 87 data blocks need recovery
Started redo application at Thread 1: logseq 98, block 1523
Completed redo application of 0.34MB
Completed crash recovery at
  Thread 1: logseq 98, block 2371, scn 2098905
  87 data blocks read, 87 data blocks written, 412 redo k-bytes read
SMON: enabling tx recovery
```

복구 시간 조절 — FAST_START_MTTR_TARGET

- 인스턴스 복구 시간 $\propto$ 마지막 체크포인트 이후 쌓인 리두의 양
- 체크포인트가 잦으면 복구는 빠르나 평상시 I/O 부하가 커진다
- FAST_START_MTTR_TARGET으로 목표 초를 지정
 - 300으로 두면 복구를 300초 안에 끝내도록 체크포인트 빈도를 자동 조절
 - 0이면 자동 조절하지 않음
- v\$instance_recovery로 현재 예상 복구 시간 확인
- 이 파라미터는 미디어 복구 시간과 무관하다

참고:

v\$instance_recovery의 ESTIMATED_MTTR이 현재 예상 복구 초, TARGET_MTTR이 목표값이다. TARGET이 0이면 자동 조절이 꺼진 상태다.

1.5

미디어 복구와 복구 유형

- 파일 자체가 유실·손상된 경우 — DBA가 직접 수행
- 복원(restore)과 복구(recover)는 다른 작업
- 완전 복구의 성립 조건 세 가지

인스턴스 복구 vs 미디어 복구

- 인스턴스 복구
 - 전제: 파일은 온전하고 메모리 내용만 유실
 - 재료: 온라인 리두 로그 + 언두
 - 수행: SMON이 자동, DBA 개입 없음
 - 결과: 데이터 손실 없음
- 미디어 복구
 - 전제: 파일 자체가 유실되거나 손상
 - 재료: 백업본 + 아카이브 로그 + 온라인 리두 로그
 - 수행: DBA가 RESTORE와 RECOVER를 직접
 - 결과: 조건에 따라 데이터 손실 가능

복원(restore)과 복구(recover)

- 복원 — 백업본을 제자리에 되돌려 놓는 것
- 복구 — 그 위에 리두를 적용해 최신 상태로 끌어올리는 것
- 복원만 하고 복구하지 않으면 백업 시점 데이터로 되돌아간다
- 복원 없이 복구만 하는 경우도 있다
 - 파일은 있는데 뒤쳐져 있을 때 — 예: 오래 OFFLINE 되어 있던 데이터 파일
- 두 용어를 섞어 쓰면 절차를 잘못 밟게 된다
 - '복구했는데 데이터가 옛날 것'이라는 상황은 복원만 하고 끝낸 경우다

완전 복구와 불완전 복구

- 완전 복구(complete recovery) — 장애 직전 시점까지, 데이터 손실 없음
- 성립 조건 세 가지가 모두 필요하다
 - 복구 대상 파일의 백업본
 - 백업 시점부터 장애 시점까지의 모든 리두
 - 현재 상태를 담은 제어 파일
- 불완전 복구(incomplete recovery) — 특정 과거 시점까지, 이후 데이터 손실
- 불완전 복구를 하게 되는 두 가지 이유
 - 어쩔 수 없는 경우 — 아카이브 로그 삭제, NOARCHIVELOG 모드
 - 의도적인 경우 — 사용자 오류를 되돌리려 실수 직전으로 이동

실무 시나리오:

판정 순서는 ① LOG_MODE 확인 ② 유실 파일에 CURRENT 리두가 포함되는지 ③ 백업본 SCN과 아카이브 SCN 구간이 이어지는지다.

RESETLOGS와 불완전 복구 종료 지점

- 불완전 복구 후에는 반드시 RESETLOGS로 열어야 한다
 - 리두 로그 시퀀스가 1로 초기화되고 새 인카네이션이 시작된다
 - 이전 백업본과 아카이브 로그는 그대로 쓸 수 없게 된다
 - 따라서 직후에 전체 백업을 다시 받아야 한다
- 종료 지점 지정 방식 네 가지
 - UNTIL CANCEL — 하나씩 적용하다 사용자가 CANCEL 입력한 지점에서 중단
 - UNTIL TIME — 지정 시각까지
 - UNTIL CHANGE(SCN) — 지정 SCN 직전까지
 - UNTIL SEQUENCE — 지정 로그 시퀀스 직전까지

NOARCHIVELOG vs ARCHIVELOG

- NOARCHIVELOG
 - 아카이브 로그 없음 — 온라인 리두가 순환하며 덮어쓰인다
 - 백업은 정상 종료 후에만, Hot Backup 불가
 - 복구는 백업 시점 복귀가 기본, 리두가 남아 있으면 그만큼 더
 - '운이 좋으면 완전 복구' — 정책이라 부를 수 없다
- ARCHIVELOG
 - 로그 스위치마다 아카이브 생성 — 리두가 영구 보존
 - Hot Backup 가능, 테이블스페이스/데이터 파일 단위 부분 복구 가능
 - 백업본과 아카이브가 이어지는 한 완전 복구 가능

장애 유형에 따른 복구 전략

- 일반 TS 데이터 파일 유실 → ARCHIVELOG면 OFFLINE·복원·복구·ONLINE (무중단)
- SYSTEM / UNDO 데이터 파일 유실 → MOUNT 상태에서 복원·복구 후 OPEN
- 제어 파일 전부 유실 → 백업본 복사 또는 CREATE CONTROLFILE 재생성
- CURRENT 리두 로그 유실 → 완전 복구 불가, 불완전 복구 후 RESETLOGS
- INACTIVE 리두 로그 유실 → CLEAR LOGFILE로 재생성, 손실 없음
- 블록 단위 손상 → Block Media Recovery, 파일 전체 복원 불필요 (16장)
- 사용자 오류 → Flashback 계열 (18장), TRUNCATE는 RMAN Table Recovery

Best Practice:

복구 작업을 시작하기 전에 손상된 현재 상태를 그대로 복사해 둔다. 제어 파일 재생성이나 RESETLOGS처럼 되돌릴 수 없는 작업 직전에는 예외 없이 수행한다.

1.6

진단 정보 — Alert Log와 ADR

- ADR은 진단 정보를 모아 두는 파일 기반 저장소
- alert log가 장애 진단의 출발점
- adrci로 필터링 조회와 인시던트 확인

ADR 디렉터리 구조

- ADR(Automatic Diagnostic Repository) — 진단 정보 파일 저장소
- 위치는 DIAGNOSTIC_DEST가 가리키며 기본값은 \$ORACLE_BASE
- 실제 경로: \$ORACLE_BASE/diag/rdbms/<db_name>/<SID>
- 주요 하위 디렉터리
 - trace — alert_<SID>.log와 각종 트레이스 파일
 - alert — XML 형식 alert log, adrci가 읽는 파일
 - incident — 심각한 오류 시 자동 생성되는 덤프
 - hm — Health Monitor 결과와 복구 스크립트, 16장 DRA가 사용

adrci로 alert log 조회

장애 시각 전후의 ORA 오류만 필터링해서 확인

```
# adrci로 alert log의 최근 오류 확인
[oracle@oel7v9r1 ~]$ adrci

ADRCI: Release 19.0.0.0.0 - Production
ADR base = "/u01/app/oracle"

adrci> SHOW HOMES
ADR Homes:
diag/rdbms/orcl/orcl

adrci> SET HOME diag/rdbms/orcl/orcl
adrci> SHOW ALERT -P "MESSAGE_TEXT LIKE '%ORA-01110%'" -TERM

2025-05-06 10:18:52.417000 +09:00
ORA-01110: data file 7: '/u01/app/oracle/oradata/ORCL/users01.dbf'
ORA-01157: cannot identify/lock data file 7 - see DBWR trace file

adrci> EXIT
```

퀴즈 1

인스턴스 복구와 미디어 복구

SHUTDOWN ABORT로 종료한 데이터베이스를 STARTUP 했다. 이때 수행되는 복구에 대한 설명으로 옳은 것을 모두 고르시오.

-
- (a) 백업본이 필요하다
 - (b) SMON이 자동 수행한다
 - (c) 데이터 손실이 없다
 - (d) DBA가 RECOVER 명령을 내려야 한다
 - (e) 온라인 리두 로그를 사용한다

퀴즈 2

리두 로그 그룹의 상태

리두 로그 그룹이 유실되었을 때 CLEAR LOGFILE로 재생성해도 데이터 손실이 없는 상태는?

-
- (a) CURRENT
 - (b) ACTIVE
 - (c) INACTIVE
 - (d) UNUSED

퀴즈 3

완전 복구의 성립 조건

완전 복구가 가능하기 위해 반드시 필요한 것을 모두 고르시오.

-
- (a) 복구 대상 파일의 백업본
 - (b) 백업 이후 장애 시점까지의 모든 리두
 - (c) ARCHIVELOG 모드
 - (d) RESETLOGS 옵션
 - (e) 현재 상태의 제어 파일

퀴즈 4

SCN 비교

MOUNT 상태에서 v\$database의 CHECKPOINT_CHANGE#는 2098460인데 file# 7은 2097189였다. 옳은 해석은?

-
- (a) 모든 파일이 일관성 있는 상태다
 - (b) file# 7만 뒤처져 있어 리두 적용이 필요하다
 - (c) 제어 파일이 손상되었다
 - (d) file# 7을 삭제하고 재생성해야 한다

퀴즈 5

커밋과 데이터 파일

COMMIT이 반환된 직후 보장되는 것은?

-
- (a) 변경된 블록이 데이터 파일에 기록되었다
 - (b) 리두 엔트리가 리두 로그 파일에 기록되었다
 - (c) 체크포인트가 완료되었다
 - (d) 아카이브 로그가 생성되었다

퀴즈 6

복원과 복구

'복원(restore)'과 '복구(recover)'의 구분으로 옳은 것은?

-
- (a) 복원은 리두 적용, 복구는 파일 되돌리기다
 - (b) 복원은 백업본을 제자리에 되돌리는 것, 복구는 그 위에 리두를 적용하는 것이다
 - (c) 두 용어는 같은 작업을 가리킨다
 - (d) 복원은 ARCHIVELOG에서만, 복구는 NOARCHIVELOG에서만 쓴다

요약 (Summary)

- 장애 6유형 중 백업본이 필요한 것은 미디어 실패뿐 — 사용자 오류는 Flashback, 인스턴스 실패는 자동 복구
- 복구 설계는 RPO·RTO에서 출발 — RPO 0이면 ARCHIVELOG와 다중화가 필수
- 복구 관여 파일: 데이터/제어/온라인 리두/아카이브/파라미터 — 제어 파일과 리두는 다중화로 예방
- SCN이 복구 판단 기준 — 제어 파일과 데이터 파일 헤더의 값이 일치하면 일관성 있는 상태
- 리두는 롤포워드, 언두는 롤백 — 커밋은 리두 기록만 보장하므로 데이터 파일은 항상 과거
- 인스턴스 복구는 자동·무손실, 미디어 복구는 DBA 수행·손실 가능
- 완전 복구 = 백업본 + 전체 리두 + 현재 제어 파일, 하나라도 없으면 불완전 복구 후 RESETLOGS
- 진단은 v\$instance STATUS → v\$database OPEN_MODE → alert log 순

실습 1 개요

- 목적: 복구 작업이 아니라, 이후 모든 시나리오에서 쓸 확인 절차를 익힌다
- 실습 1-1 백업 대상 파일 확인 — 통합 조회, 다중화 점검, TEMP의 예외적 성격
- 실습 1-2 인스턴스 복구 동작 확인 — SHUTDOWN ABORT 후 crash recovery 관찰
- 실습 1-3 체크포인트와 SCN 흐름 추적 — 커밋·로그 스위치의 영향 비교
- 실습 1-4 백업 스크립트 생성 — 조회 결과로 OS 복사 명령 자동 생성, PFILE·CF 트레이스 확보
- 실습 1-5 Alert Log와 ADR 확인 — v\$diag_info로 경로 확인 후 adrci 필터 조회

참고:

실습 환경은 non-CDB ORCL이며 SQL*Plus에 SYSDBA로 접속해 수행한다. 1-2의 SHUTDOWN ABORT는 파일을 손상시키지 않으므로 안전하나, 다른 사용자가 접속한 환경에서는 수행하지 않는다.